

CHILDSHIELD CLIMATE AI

Guia Técnico de Implementação
Arquitectura · Fluxos · Código · Plano de 5 Dias

Stack: Laravel · Next.js · PostgreSQL+PostGIS · Africa's Talking · Baileys · OpenAI · Mapbox

Moçambique · 2025

Índice

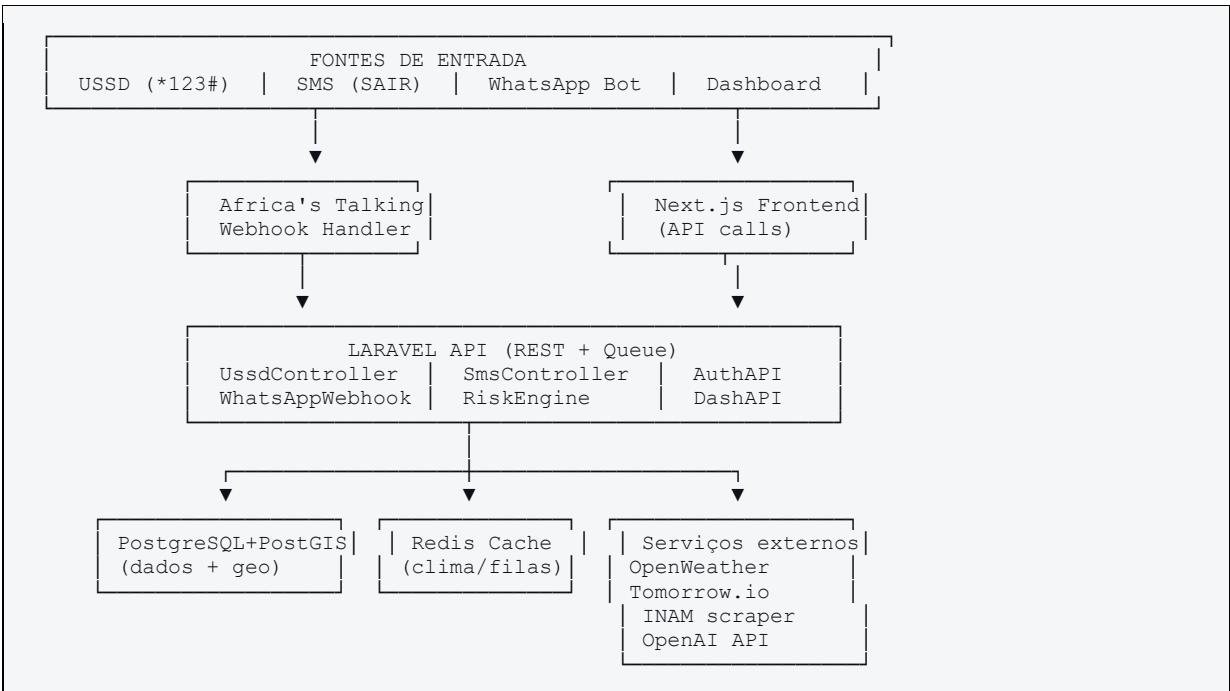
1. Arquitectura do Sistema e Diagrama de Componentes
2. Implementação USSD (Africa's Talking + Laravel)
3. Implementação SMS (Alertas automáticos)
4. Implementação WhatsApp Bot (Baileys / WABA)
5. Motor de Risco Climático (RiskEngineService)
6. Integrações Climáticas (OpenWeather · Tomorrow.io · INAM)
7. Dashboard Web (Next.js · Mapbox · API REST)
8. Base de Dados e Migrações (PostgreSQL + PostGIS)
9. Segurança, Autenticação e Protecção de Dados
10. Plano de Execução de 5 Dias com Ferramentas de IA

1. Arquitectura do Sistema

Visão geral dos componentes

Componente	Tecnologia	Responsabilidade
API Backend	Laravel 11 (PHP 8.3)	REST API, filas, motor de risco, cron jobs
Frontend Dashboard	Next.js 14 + Tailwind CSS	SPA/SSR para clínicas, ONGs e governo
Base de dados	PostgreSQL 16 + PostGIS 3.4	Dados relacionais + geoespaciais
Cache & Filas	Redis 7	Cache de dados climáticos; filas de envio (Laravel Queue)
USSD Gateway	Africa's Talking USSD API	Recepciona sessões USSD de Vodacom/Movitel/Tmcel
SMS Gateway	Africa's Talking SMS API	Envio massivo com relatório de entrega
WhatsApp (MVP)	Baileys (Node.js library)	Bot WhatsApp sem aprovação Meta; sessão WS
WhatsApp (prod)	360dialog / Meta WABA	API oficial; obrigatório para UNICEF
Clima	OpenWeather + Tomorrow.io + INAM	Dados meteorológicos em tempo real e previsão
IA / NLG	OpenAI GPT-4o-mini	Geração de mensagens adaptadas; configurável
Mapas	Mapbox GL JS	Mapa interactivo de risco no dashboard
Deploy	Railway / DigitalOcean App Platform	CI/CD automático a partir do GitHub

Fluxo de dados de alto nível



2. Implementação USSD (Africa's Talking + Laravel)

Configuração Africa's Talking

O Africa's Talking fornece um endpoint webhook que o Laravel expõe. A cada pressão de tecla da família, a AT envia um POST com os parâmetros da sessão.

Variáveis de ambiente (.env)

```
AT_API_KEY=your_africastalking_api_key
AT_USERNAME=childshield
AT_USSD_CODE=*123#
AT_SENDER_ID=ChildShield
```

Controller USSD (Laravel)

```
// app/Http/Controllers/UssdController.php

class UssdController extends Controller
{
    public function handle(Request $request): Response
    {
        $sessionId = $request->input('sessionId');
        $phoneNumber = $request->input('phoneNumber');
        $text = $request->input('text', ''); // input acumulado

        $parts = explode('*', $text);
        $level = count(array_filter($parts, fn($p) => $p !== ''));

        $user = User::where('phone number', $phoneNumber)->first();
        $hasActiveSub = $user && $user->subscription_active;

        // Nível 0: menu principal
        if ($text === '') {
            $menu = "Bem-vindo ao ChildShield\n"
                . "1. Registrar família\n"
                . "2. Ver alerta actual\n"
                . "3. Alterar localização\n"
                . "4. Escolher idioma\n"
                . ($hasActiveSub ? "5. Cancelar subscrição\n" : '')
                . "0. Sair";
            return response($menu)->header('Content-Type', 'text/plain');
        }

        $choice = $parts[0] ?? '';

        return match ($choice) {
            '1' => app(UssdRegistrationFlow::class)->handle($parts, $phoneNumber),
            '2' => app(UssdAlertFlow::class)->handle($phoneNumber),
            '3' => app(UssdLocationFlow::class)->handle($parts, $phoneNumber),
            '4' => app(UssdLanguageFlow::class)->handle($parts, $phoneNumber),
            '5' => $hasActiveSub
                ? app(UssdCancelFlow::class)->handle($parts, $phoneNumber)
                : response("Opção inválida.", 200),
            default => response("END Opção inválida.", 200),
        };
    }
}
```

Fluxo de Registo (UssdRegistrationFlow)

```
// Passo por passo – cada * na string 'text' representa um nível
// text='1'           → escolheu Registrar
// text='1*2'         → escolheu Província 2 (Sofala)
// text='1*2*3'       → escolheu Distrito 3
// ...

public function handle(array $parts, string $phone): Response
```

```

{
    return match (count($parts)) {
        1 => response($this->provincesMenu()),
        2 => response($this->districtsMenu((int)$parts[1])),
        3 => response("Número de crianças em casa:\n1. Uma\n2. Duas\n3. Três\n4. Quatro
ou mais"),
        4 => response("Existe grávida em casa?\n1. Sim\n2. Não"),
        5 => $parts[4] === '1'
            ? response("Semanas de gestação?\n1. 1-12 semanas\n2. 13-24 semanas\n3.
25+ semanas")
            : $this->selectLanguage($parts, $phone),
        6 => $parts[4] === '1'
            ? $this->selectLanguage($parts, $phone)
            : $this->confirmAndSave($parts, $phone),
        7 => $this->confirmAndSave($parts, $phone),
        default => response('END Erro. Tente novamente.'),
    };
}

private function confirmAndSave(array $parts, string $phone): Response
{
    // Mapear escolhas → dados reais
    $location = Location::findByProvDistrict(
        $this->provinceMap[$parts[1]],
        $this->districtMap[$parts[1]][$parts[2]]
    );

    User::updateOrCreate(
        ['phone number' => $phone],
        ['location id' => $location->id, 'subscription active' => true,
        'consent status' => 'accepted', 'channel' => 'ussd']
    );

    // Disparar SMS de confirmação de forma assíncrona
    SendSmsJob::dispatch($phone, __('sms.welcome', [], $language));

    return response('END Registo feito! Receberá alertas ChildShield. Para cancelar:
SAIR.');
```

Timeout USSD em Moçambique

As operadoras moçambicanas têm um timeout de sessão USSD de 90–120 segundos. O fluxo de registo deve ter no máximo 7 interações. Fluxos longos devem ser divididos — o SMS de confirmação pode pedir dados adicionais opcionais.

3. Implementação SMS — Alertas Automáticos

Arquitectura de envio assíncrono

O envio de SMS é feito de forma assíncrona via Laravel Queue (driver: Redis). Um cron job corre de hora em hora, calcula o risco e enfileira jobs de envio. Evita timeout no processo principal e permite retentar falhas.

Cron Job — ClimateAlertScheduler

```

// app/Console/Kernel.php
protected function schedule(Schedule $schedule): void
{
    // Recolha de dados climáticos a cada hora
    $schedule->job(new FetchClimateDataJob)->hourly();

    // Calcular risco e enviar alertas de manhã e ao fim da tarde
    $schedule->job(new CalculateRiskAndAlertJob)->twiceDaily(7, 17);
}
```

```
// Relatório diário para o dashboard
$schedule->job(new GenerateDailyReportJob)->dailyAt('06:00');
}
```

Job de envio de alertas (CalculateRiskAndAlertJob)

```
class CalculateRiskAndAlertJob implements ShouldQueue
{
    public function handle(RiskEngineService $engine, MessageGeneratorService $msg)
    {
        $locations = Location::with('latestClimateData')->get();

        foreach ($locations as $location) {
            $scores = $engine->calculateAllRisks($location);

            foreach ($scores as $type => $score) {
                $level = $engine->scoreToLevel($score);
                if ($level === 'low') continue; // Não enviar para risco baixo

                $previousScore = RiskScore::latest($location, $type);
                if ($previousScore && $previousScore->risk level === $level) continue; //
Sem mudança

                // Guardar novo score
                RiskScore::create([..., 'risk level' => $level, 'score' => $score]);

                // Gerar mensagem (IA ou template)
                $message = $msg->generate($location, $type, $level);

                // Enfileirar envios por utilizador da zona
                $users = User::subscribedInLocation($location->id)->get();
                foreach ($users as $user) {
                    SendAlertJob::dispatch($user, $message, $type, $level)
                        ->onQueue('alerts');
                }
            }
        }
    }
}
```

Envio via Africa's Talking SDK

```
// composer require africastalking/africastalking

class SmsService
{
    private AfricasTalking $at;

    public function __construct()
    {
        $this->at = new AfricasTalking(
            config('services.africastalking.username'),
            config('services.africastalking.api_key')
        );
    }

    public function send(string $phone, string $message, string $senderId =
'ChildShield'): array
    {
        try {
            $sms = $this->at->sms();
            $result = $sms->send([
                'to' => $phone,
                'message' => $message,
                'from' => $senderId,
            ]);
            return ['success' => true, 'result' => $result];
        }
    }
}
```

```

    } catch (Exception $e) {
        Log::error('SMS failed: ' . $e->getMessage());
        return ['success' => false, 'error' => $e->getMessage()];
    }
}
}

```

4. Implementação WhatsApp Bot

MVP: Baileys (Node.js)

Baileys é uma biblioteca Node.js open-source que implementa o protocolo WhatsApp Web. Ideal para MVP e testes. Não requer aprovação da Meta nem Business Account.

⚠ Atenção para produção

Baileys/WhatsApp Web.js viola os Termos de Serviço do WhatsApp. Para a apresentação UNICEF e para produção, migrar para WhatsApp Business API oficial (360dialog ou Twilio). O código de negócio (fluxo do bot) é reutilizável — apenas a camada de transporte muda.

Arquitetura do Bot WhatsApp

```

// whatsapp-service/ (serviço Node.js independente, comunica com Laravel via HTTP)

// package.json deps:
// @whiskeysockets/baileys, axios, express, qrcode-terminal

import makeWASocket, { useMultiFileAuthState } from '@whiskeysockets/baileys';
import axios from 'axios';

const { state, saveCreds } = await useMultiFileAuthState('auth info');

const sock = makeWASocket({ auth: state });
sock.ev.on('creds.update', saveCreds);

sock.ev.on('messages.upsert', async ({ messages }) => {
  for (const msg of messages) {
    if (!msg.message || msg.key.fromMe) continue;

    const from      = msg.key.remoteJid;
    const text      = msg.message.conversation
      || msg.message.extendedTextMessage?.text || '';

    // Enviar para Laravel processar o fluxo do bot
    const { data } = await axios.post('http://laravel-api/api/whatsapp/message', {
      phone: from.replace('@s.whatsapp.net', ''),
      message: text.trim(),
    });

    // Responder com o que o Laravel retornou
    await sock.sendMessage(from, { text: data.reply });
  }
});

```

Controller WhatsApp no Laravel (WhatsAppController)

```

class WhatsAppController extends Controller
{
    public function handleMessage(Request $request): JsonResponse
    {

```

```

{
    $phone = $request->input('phone');
    $text = strtolower(trim($request->input('message')));

    $session = WhatsAppSession::firstOrCreate(
        ['phone' => $phone],
        ['step' => 'main_menu', 'data' => []]
    );

    // Opt-out global
    if (in_array($text, ['sair', 'stop', 'cancelar', '0'])) {
        User::where('phone number', $phone)->update(['subscription active' =>
false]);
        $session->delete();
        return response()->json(['reply' => 'Subscrição cancelada. Até breve!']);
    }

    $reply = app(WhatsAppFlowService::class)->process($session, $text);

    return response()->json(['reply' => $reply]);
}
}

```

WhatsAppFlowService — máquina de estados

```

class WhatsAppFlowService
{
    public function process(WhatsAppSession $session, string $input): string
    {
        return match ($session->step) {
            'main menu' => $this->mainMenu($session, $input),
            'register province' => $this->registerProvince($session, $input),
            'register district' => $this->registerDistrict($session, $input),
            'register children' => $this->registerChildren($session, $input),
            'register pregnant' => $this->registerPregnant($session, $input),
            'register language' => $this->registerLanguage($session, $input),
            'report symptoms' => $this->reportSymptoms($session, $input),
            default => $this->mainMenu($session, ''),
        };
    }

    private function mainMenu(WhatsAppSession $session, string $input): string
    {
        if ($input === '') {
            return "Olá! Sou o ChildShield ☑️\n"
                . "1. Registrar a minha família\n"
                . "2. Ver alerta da minha zona\n"
                . "3. Dicas para proteger crianças\n"
                . "4. Reportar sintomas\n"
                . "5. Unidades sanitárias próximas\n"
                . "0. Sair";
        }
        return match ($input) {
            '1' => $this->startRegistration($session),
            '2' => $this->showAlert($session),
            '3' => $this->showTips($session),
            '4' => $this->startSymptomReport($session),
            '5' => $this->showNearbyClinic($session),
            default => "Opção inválida. Responda com 1-5 ou 0 para sair.",
        };
    }
}

```

5. Motor de Risco Climático (RiskEngineService)

Lógica de cálculo

O motor recebe os dados climáticos mais recentes de uma localização e os scores estáticos da mesma, produzindo um score 0–100 por tipo de risco. Score ≥ 60 gera alertas automáticos.

```
// app/Services/RiskEngineService.php

class RiskEngineService
{
    public function calculateAllRisks(Location $location): array
    {
        $climate = $location->latestClimateData;
        if (!$climate) return [];

        return [
            'heat'          => $this->heatRisk($climate),
            'malaria'       => $this->malariaRisk($climate, $location),
            'diarrhea'      => $this->diarrheaRisk($climate, $location),
            'respiratory'   => $this->respiratoryRisk($climate, $location),
        ];
    }

    private function heatRisk(ClimateData $c): int
    {
        $base = match (true) {
            $c->temperature >= 40 => 100,
            $c->temperature >= 36 => 70,
            $c->temperature >= 32 => 40,
            default                => 0,
        };
        // Modificador: zona costeira ou árida
        return min(100, $base + ($c->location->is_coastal ? 10 : 0));
    }

    private function malariaRisk(ClimateData $c, Location $loc): int
    {
        $base = 0;
        if ($c->rainfall_24h >= 20 && $c->humidity >= 70) $base = 60;
        elseif ($c->rainfall_24h >= 10 && $c->humidity >= 60) $base = 35;

        // Peso estático da zona (0-40 definido no dashboard)
        $static = $loc->malaria_risk_static ?? 0;

        return min(100, (int)(($base + $static) * ($static > 20 ? 1.3 : 1.0)));
    }

    private function diarrheaRisk(ClimateData $c, Location $loc): int
    {
        $base = 0;
        if ($c->rainfall_24h >= 30 && $c->temperature > 28) $base = 55;
        elseif ($c->rainfall_24h >= 15) $base = 30;

        $sanitation = 100 - ($loc->sanitation_score ?? 50); // score baixo = risco alto
        $multiplier = $loc->sanitation_score < 40 ? 1.4 : 1.0;

        return min(100, (int)(($base + $sanitation * 0.3) * $multiplier));
    }

    private function respiratoryRisk(ClimateData $c, Location $loc): int
    {
        $base = 0;
        if ($c->wind_speed > 30 && $c->humidity < 40) $base = 50;
        elseif ($c->air_quality_index > 100) $base = 40;

        return min(100, $base + ($loc->is_urban ? 20 : 0));
    }

    public function scoreToLevel(int $score): string
    {
        return match (true) {
            $score >= 85 => 'critical',
            $score >= 60 => 'high',
            $score >= 30 => 'medium',
            default      => 'low',
        };
    }
}
```

```
}
}
```

6. Integrações Climáticas

Estratégia de conjugação de fontes

Fonte	Dados fornecidos	Frequência	Custo
OpenWeatherMap	Temp, humidade, vento, chuva actual	Hora a hora	Grátis até 1000/dia
Tomorrow.io	Previsão 5 dias, precipitação, humidade	6/6 horas	Grátis 500/dia
INAM (scraping)	Alertas nacionais (ciclones, cheias)	Diário	Gratuito (público)

FetchClimateDataJob — Laravel

```
class FetchClimateDataJob implements ShouldQueue
{
    public function handle(OpenWeatherService $ow, TomorrowService $tmrw)
    {
        $locations = Location::all();

        foreach ($locations->chunk(50) as $chunk) {
            foreach ($chunk as $loc) {
                // Tentar OpenWeather primeiro (mais chamadas grátis)
                $data = $ow->getCurrentWeather($loc->latitude, $loc->longitude);

                if (!$data) {
                    // Fallback para Tomorrow.io
                    $data = $tmrw->getCurrent($loc->latitude, $loc->longitude);
                }

                if ($data) {
                    ClimateData::create([
                        'location id' => $loc->id,
                        'temperature' => $data['temp'],
                        'rainfall 24h' => $data['rain'],
                        'humidity' => $data['humidity'],
                        'wind_speed' => $data['wind_speed'],
                        'source' => $data['source'],
                        'date time' => now(),
                    ]);
                }
            }
            sleep(1); // Rate limiting
        }
    }
}
```

OpenWeatherService

```
class OpenWeatherService
{
    private string $apiKey;
    private string $baseUrl = 'https://api.openweathermap.org/data/2.5/weather';

    public function getCurrentWeather(float $lat, float $lon): ?array
```

```

{
  $cacheKey = "owm:${$lat}:${$lon}";

  return Cache::remember($cacheKey, 3600, function () use ($lat, $lon) {
    $response = Http::get($this->baseUrl, [
      'lat' => $lat,
      'lon' => $lon,
      'appid' => $this->apiKey,
      'units' => 'metric',
    ]);

    if ($response->failed()) return null;

    $d = $response->json();
    return [
      'temp' => $d['main']['temp'],
      'humidity' => $d['main']['humidity'],
      'wind speed' => $d['wind']['speed'] * 3.6, // m/s → km/h
      'rain' => $d['rain']['1h'] ?? 0,
      'source' => 'openweather',
    ];
  });
}

```

7. Dashboard Web (Next.js + Mapbox)

Estrutura de páginas e rotas

Página	Rota Next.js	Componentes principais
Login	/login	Formulário + validação JWT
Dashboard principal	/dashboard	MapboxRiskMap, RiskCards, AlertFeed
Mapa detalhado	/dashboard/map	MapboxGL full, filtros por tipo/nível
Famílias registadas	/dashboard/families	Tabela, filtros por zona, exportar
Gestão de campanhas	/dashboard/campaigns	Criar SMS, agendar, ver estatísticas
Relatórios	/dashboard/reports	Filtros, preview, exportar PDF/Excel
Administração	/admin	Utilizadores, motor de risco, logs

Componente de Mapa de Risco (Mapbox GL + GeoJSON)

```

// components/RiskMap.tsx

const RiskMap: React.FC<{ riskData: RiskGeoJSON }> = ({ riskData }) => {
  const mapRef = useRef<MapRef>(null);

  const riskColors = {
    critical: '#C0392B',
    high: '#E67E22',
    medium: '#F1C40F',
    low: '#27AE60',
  };

  return (
    <Map
      ref={mapRef}
      mapboxAccessToken={process.env.NEXT_PUBLIC_MAPBOX_TOKEN}
      initialState={{ longitude: 35.0, latitude: -18.5, zoom: 5 }}
    />
  );
}

```

```

        style={{ width: '100%', height: '60vh' }}
        mapStyle="mapbox://styles/mapbox/light-v11"
    >
        <Source id="risk" type="geojson" data={riskData}>
            <Layer
                id="risk-fill"
                type="fill"
                paint={{
                    'fill-color': ['get', 'color'],
                    'fill-opacity': 0.6,
                }}
            />
            <Layer
                id="risk-outline"
                type="line"
                paint={{ 'line-color': '#FFFFFF', 'line-width': 1 }}
            />
        </Source>
    </Map>
);
};

```

API Laravel para o Dashboard

```

// routes/api.php

Route::prefix('dashboard')->middleware(['auth:sanctum', 'role:clinic,ong,admin'])-
>group(function () {
    Route::get('/risk-map', [DashboardController::class, 'riskMap']);
    Route::get('/active-alerts', [DashboardController::class, 'activeAlerts']);
    Route::get('/kpis', [DashboardController::class, 'kpis']);
    Route::get('/families', [FamilyController::class, 'index']);
    Route::post('/campaigns', [CampaignController::class, 'store']);
    Route::get('/reports', [ReportController::class, 'export']);
});

// Endpoint GeoJSON para o mapa
// GET /api/dashboard/risk-map?type=malaria&level=high
// Retorna FeatureCollection com polígonos de distritos coloridos por risco

```

8. Base de Dados — Migrações PostgreSQL + PostGIS

Migration: locations (com PostGIS)

```

// database/migrations/create locations table.php
Schema::create('locations', function (Blueprint $table) {
    $table->id();
    $table->string('province');
    $table->string('district');
    $table->string('locality')->nullable();
    $table->decimal('latitude', 10, 7);
    $table->decimal('longitude', 10, 7);
    // Coluna PostGIS — executar manualmente ou via DB::statement
    // DB::statement('ALTER TABLE locations ADD COLUMN geom GEOMETRY(Point, 4326)');
    // DB::statement('CREATE INDEX idx_locations_geom ON locations USING GIST(geom)');
    $table->integer('malaria_risk_static')->default(0); // 0-40
    $table->integer('sanitation_score')->default(50); // 0-100 (100=bom)
    $table->integer('flood_risk')->default(0); // 0-40
    $table->boolean('is_coastal')->default(false);
    $table->boolean('is_urban')->default(false);
    $table->timestamps();
});

```

Migration: users

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('phone_number')->unique();
    $table->enum('channel', ['ussd', 'sms', 'whatsapp']);
    $table->string('language', 10)->default('pt');
    $table->foreignId('location_id')->nullable()->constrained();
    $table->enum('consent status', ['accepted', 'declined', 'pending'])-
    >default('pending');
    $table->boolean('subscription_active')->default(true);
    $table->timestamp('subscribed_at')->nullable();
    $table->timestamp('unsubscribed_at')->nullable();
    $table->timestamps();
});
```

Consulta espacial — famílias num raio de 10km de uma clínica

```
-- PostGIS: familias a menos de 10km da clínica (lat -25.9, lon 32.6)
SELECT u.phone number, l.district
FROM users u
JOIN locations l ON u.location_id = l.id
WHERE ST_DWithin(
    l.geom,
    ST_SetSRID(ST_MakePoint(32.6, -25.9), 4326)::geography,
    10000 -- 10km em metros
)
AND u.subscription_active = true;
```

9. Segurança e Autenticação

Autenticação do Dashboard

- Laravel Sanctum para tokens API (dashboard Next.js)
- RBAC com roles: admin | government | ong | clinic — via spatie/laravel-permission
- JWT tokens com expiração de 8h; refresh token com 7 dias
- Rate limiting: 60 req/min por IP nos endpoints públicos; 300/min para autenticados

Protecção de dados (famílias)

- phone_number cifrado em repouso com AES-256 (Laravel Encrypted Cast)
- Anonimização automática em exports: phone → hash(sha256(phone + salt))
- Soft delete para cancelamentos — dados guardados 90 dias para auditoria, depois eliminados
- Logs de actividade com laravel-activitylog: quem exportou, quando, que dados

Infraestrutura

- HTTPS obrigatório (certificado Let's Encrypt via Caddy ou Nginx)
- Variáveis sensíveis em .env — nunca no repositório Git
- Docker Compose para ambiente local; Railway/DigitalOcean para staging e produção
- Backups automáticos PostgreSQL: pg_dump diário para S3/Backblaze B2

10. Plano de Execução de 5 Dias

Ferramentas de IA no desenvolvimento

O plano integra Cursor IDE (autocomplete contextual + chat), Claude API (geração de código e revisão), v0.dev (prototipagem de componentes React), GitHub Copilot (pares de programação), e ChatGPT/Claude para documentação automática. O objectivo é multiplicar a velocidade de cada dev sem sacrificar qualidade.

DIA 1 — Fundações e Setup (Setup Day)

Hora	Tarefa	Ferramentas de IA
09:00–10:30	Setup do repositório GitHub (monorepo: laravel-api + next-dashboard + whatsapp-bot); Docker Compose com Laravel, PostgreSQL+PostGIS, Redis	Cursor: gerar docker-compose.yml e .env.example; Claude: arquitectura inicial
10:30–12:30	Todas as migrações do Laravel (users, households, locations, climate_data, risk_scores, alerts, symptom_reports); seed de províncias/distritos de Moçambique	Cursor+Copilot: gerar migrações; Claude: seed de dados geográficos de Moçambique em JSON
13:30–15:30	API REST básica: autenticação (Sanctum), RBAC (Spatie), endpoints de saúde e estrutura de controllers	Cursor: scaffold dos controllers; Claude: definir estrutura de rotas conforme spec
15:30–17:30	Setup Next.js: layout, sidebar, routing, Tailwind config, integração Mapbox básica; componentes base (Button, Card, Badge de risco)	v0.dev: gerar layout dashboard; Cursor: adaptar ao projecto; v0.dev para componente RiskBadge
17:30–18:00	Review do dia, commit, deploy de staging (Railway)	Claude: review do código do dia; GitHub Actions setup

Dica Dia 1 — Seed geográfico

Pedir ao Claude: 'Gera um seeder Laravel com todas as províncias e principais distritos de Moçambique em português com coordenadas lat/lon aproximadas para PostgreSQL'. Poupa 2h de pesquisa manual.

DIA 2 — Backend Core: Motor de Risco + Clima + SMS

Hora	Tarefa	Ferramentas de IA
09:00–11:00	OpenWeatherService + TomorrowService + InamScraperService; FetchClimateDataJob; testes com coordenadas de Maputo, Beira e Nampula	Cursor: implementar os services com HTTP/Cache; Claude: lógica de fallback e rate limiting
11:00–13:00	RiskEngineService completo (heat, malaria, diarrhea, respiratory) com camadas estáticas; CalculateRiskAndAlertJob; testes unitários	Cursor+Copilot: gerar testes PHPUnit; Claude: validar a lógica do motor com casos de teste reais
14:00–16:00	Africa's Talking SDK: SmsService + SendAlertJob + sistema de filas Redis; implementar opt-out via SMS (keyword SAIR)	Cursor: scaffold do job e queue worker; Claude: lógica de deduplicação de alertas
16:00–18:00	MessageGeneratorService: templates de mensagens por tipo/nível/idioma; integração OpenAI para geração dinâmica (configurável)	Claude API: gerar templates de SMS em Português, Changane e Sena; Cursor: integrar cliente OpenAI

Dica Dia 2 — Motor de risco com Claude

Partilhar as regras do motor (secção 3 da spec) ao Claude e pedir: 'Converte estas regras numa classe PHP com score 0-100, incluindo testes unitários com dados climáticos de Moçambique'. Output pronto em 10 minutos.

DIA 3 — USSD + WhatsApp Bot

Hora	Tarefa	Ferramentas de IA
09:00–11:30	UssdController + todos os fluxos (Registration, Alert, Location, Language, Cancel); sandbox Africa's Talking com números de teste; testar todos os paths	Cursor: scaffold dos flows; Claude: gerar todos os textos USSD em Português e simular edge cases
11:30–13:00	WhatsApp service Node.js: Baileys setup, auth storage, evento de mensagens, proxy para Laravel	Cursor: setup Baileys; Claude: lógica de sessão e edge cases (mensagem vazia, emoji, etc.)
14:00–16:30	WhatsAppFlowService no Laravel: todos os steps (main_menu, register, alert, tips, symptoms, clinics); WhatsAppSession model	Cursor: gerar a máquina de estados; Claude: validar fluxo de registo WhatsApp vs. USSD (consistência)
16:30–18:00	Teste end-to-end: registar família via USSD → disparar alerta → receber SMS + WhatsApp; verificar opt-out	Claude: rever o fluxo completo e identificar edge cases; Cursor: corrigir bugs rapidamente

Dica Dia 3 — Teste USSD

O Africa's Talking tem um simulador USSD no dashboard. Testar com *123# e simular todos os caminhos de registo. Para WhatsApp, usar um número de teste com Baileys antes de usar o número principal.

DIA 4 — Dashboard Frontend + API de Dashboard

Hora	Tarefa	Ferramentas de IA
09:00–11:00	Dashboard principal: RiskMap (Mapbox + GeoJSON), RiskCards (4 tipos), AlertFeed em tempo real (polling 60s); dados reais da API	v0.dev: gerar componentes RiskCard e AlertFeed; Cursor: integrar com API Laravel; Copilot: hooks React
11:00–13:00	API REST completa para dashboard: /risk-map (GeoJSON), /active-alerts, /kpis, /families; transformadores de dados para GeoJSON	Cursor+Copilot: API Resources Laravel; Claude: gerar query PostGIS para GeoJSON de distritos
14:00–16:00	Gestão de campanhas SMS no dashboard: criar campanha, seleccionar zona, agendar, ver histórico e taxa de entrega	v0.dev: gerar UI de criação de campanha; Cursor: integrar com CampaignController
16:00–18:00	Exportação de relatórios: PDF (Laravel DomPDF ou Snappy) + Excel (Maatwebsite/Excel); filtros por data, zona e tipo de risco	Cursor: setup exportadores; Claude: gerar template de relatório HTML para PDF com dados da clínica

Dica Dia 4 — GeoJSON de Moçambique

Obter o GeoJSON dos distritos de Moçambique em <https://github.com/geoafrikana/mozambique-geojson> ou OpenStreetMap. Usar o Claude para converter e limpar o GeoJSON para o formato esperado pelo Mapbox.

DIA 5 — Testes, Documentação e Deploy

Hora	Tarefa	Ferramentas de IA
09:00–11:00	Testes de integração end-to-end: registrar → alerta → SMS → WhatsApp → dashboard → export; corrigir bugs identificados	Cursor: gerar testes de integração PHPUnit e Playwright (Next.js); Claude: identificar casos edge
11:00–12:30	Ajuste e revisão das mensagens SMS/WhatsApp com equipa de saúde; validar terminologia médica e clareza das recomendações	Claude: revisar clareza e adequação cultural das mensagens para Moçambique; tradução inicial Changane
13:30–15:00	Documentação técnica: README, swagger/OpenAPI, guia de instalação Docker, CONTRIBUTING.md	Claude: gerar README completo a partir do código e specs; Cursor: gerar OpenAPI spec dos controllers
15:00–17:00	Deploy produção: Railway (API + Worker) + Vercel (Next.js) + Neon/Supabase (PostgreSQL); configurar domínio, SSL, variáveis de ambiente	Claude: gerar GitHub Actions workflow para CI/CD; Cursor: Dockerfile otimizado
17:00–18:00	Preparação demo UNICEF: criar família de teste em Sofala, disparar alerta de malária, mostrar SMS + dashboard em tempo real	Claude: gerar script de demo e FAQ técnico para parceiros

Resumo do plano — Entregáveis por dia

Dia	Foco	Entregável
1	Fundações	Repo Git + Docker + DB migrations + Auth API + Layout dashboard
2	Backend Core	Motor de risco + integrações clima + SMS automático + IA de mensagens
3	USSD + WhatsApp	Todos os fluxos USSD + Bot WhatsApp funcional end-to-end
4	Dashboard	Dashboard completo com mapa, KPIs, campanhas e exportação
5	Quality & Ship	Testes + documentação + deploy produção + demo UNICEF pronta

Ferramentas de IA recomendadas — Resumo

Cursor IDE (editor principal com IA contextual) · GitHub Copilot (pares de programação, autocompletação) · Claude claude-sonnet-4-20250514 via API (geração de código complexo, revisão, docs) · v0.dev (componentes React/Tailwind a partir de texto) · Midjourney/DALL-E (assets visuais para o dashboard, se necessário). Meta-dica: sempre que uma tarefa levar mais de 30 minutos, primeiro perguntar ao Claude como a decompor ou automatizar.